

Extended Kalman Filter

Team ID: SDCC2021001

Presenter: Sowjanya Matakala

PROJECT - 6 REPORT

Submitted by: Team Avida of *National Institute of Technology Agartala*

For: SAE Self Driving Car Challenge 2022

Extended Kalman Filter

PROJECT REPORT

For SAE India (Southern Section) SDCC

Submitted by,

Team AVIDA (NIT Agartala)

in partial fulfillment for the completion of

SAE Self-Driving Car Challenge

Under the guidance of,

Mr. Shanmugam Sivaraman

(Project Mentor)

DECLARATION

I the undersigned solemnly declare that the project report titled LANE FINDING BASIC is based on my own work carried out during the course of our study under the supervision of Mr. Shanmugam Sivaraman, Secretary, SAE India Southern section.

I assert the statements made and conclusions drawn are an outcome of my research work. I further certify that,

I. The work contained in the report is original and has been done by me under the general supervision of my supervisor.
II. The work has not been submitted to any other institution for any other degree/diploma/certificate in this university or any other university of India or abroad.
III. I have followed the guidelines provided by the university in writing the report.
IV. Whenever I have used materials (data, theoretical analysis and text) from other sources, I have given due credit to them in the references.

KeerthanaNeigapula
Hareesh M
Shivansh Mishra
Dikshita Jaraut
Sowjanya Matakala
Sumit Anand
Anawil Tiwari
Krushnakant Borude
Thanesh Bisen
Rakshit Gupta

Date – 09/06/2022

ACKNOWLEDGEMENTS

I would like to use this opportunity to thank my mentor Mr. Shanmugam Sivaraman, Secretary, SAE India Southern section. for guiding me throughout the project duration. I would also like to thank SAE India Southern Section for providing me an opportunity to work on this interesting project.

I owe gratitude to various books and online resources for the completion of this project. These resources have been mentioned in the references.

Project # 6

Extended Kalman Filter:

Objective:

The objective of project 6 is implementation of extended Kalman filter. With the provided data simulated lidar and radar measurements detecting a bicycle that travels around a vehicle. By using extended Kalman filter, we have to find lidar measurements and radar measurements to track the bicycle's position and velocity.

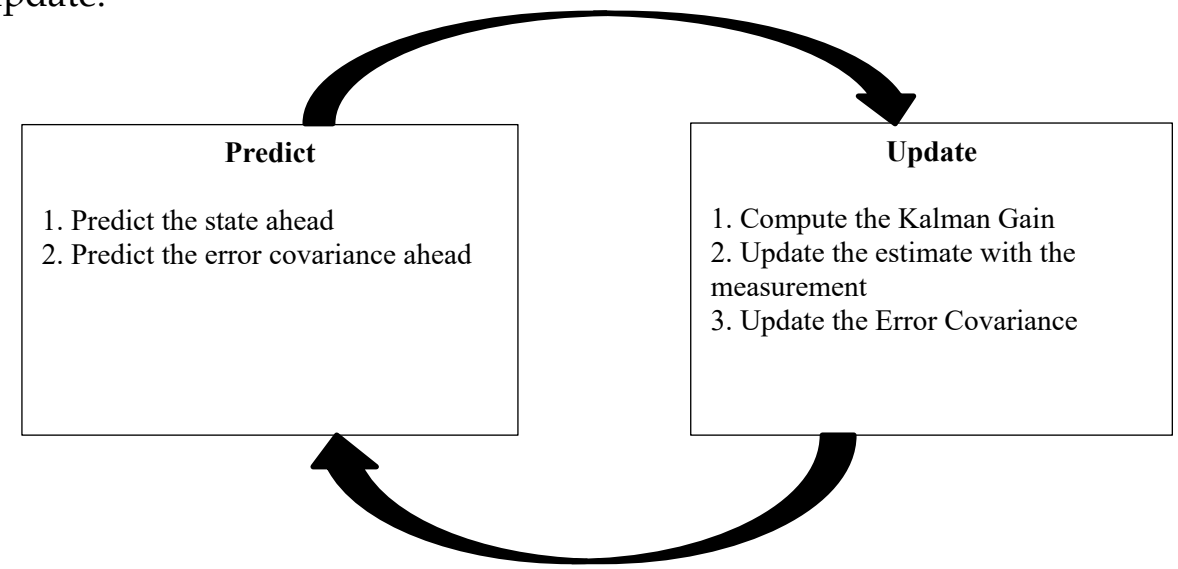
Introduction:

A self-driving car needs a map of the world around it as it drives. It must be able to track vehicles, pedestrians and other moving objects on the road continuously and to track these movements we take continuous and dynamic measurements from various sensors like Radars, Lidars etc. But there is usually a challenge that sensor measurements are uncertain due to reason which includes, but not are limited to, errors from a sensor, discrepant measurement from multiple sensors. So to get the accurate measurements by reducing the error we use an algorithm knows as *Kalman Filters*. A Kalman filter is an optimal estimation algorithm used to estimate states of a system from indirect and uncertain measurements. A Kalman filter is only defined for linear systems. If you have a nonlinear system and want to estimate system states, you need to use a nonlinear state estimator.
Kalman filter is extensively used in estimating the car position using Lidar and Radar sensors. Measurement of these sensors are not accurate as they are subject to drift or noisy. Kalman filter can be used to fuse the measurement of these sensors to find the optimal estimation of the exact position.
Sensors which is used in this project are Radar and Lidar. The **RADAR,** is

less accurate but is able to provide an estimate of both position and velocity of an object. The velocity is estimated using the Doppler Effect. Radar is able to detect objects up to 200 m from the vehicle. It is also less impervious to weather conditions. Whereas, **LIDAR** uses a laser for measurement and generates a point cloud of the world around it providing the car with fairly accurate position x and position y values. It is able to detect objects in the vicinity of the car (20–40 m) with very high accuracy. However, LIDAR is not very accurate in poor weather conditions or if the sensor gets dirty.

How does Kalman Filter work:

Kalman Filter is an iterative process that follows two steps: predict and update.



Predict: In this step the Kalman Filter predict the new values from the initial value and then predict the uncertainty/error/variance in our prediction according to the various process noise present in the system. In autonomous vehicle process noise can be understood by taking a simple example of a car moving infront of our vehicle. Our model will assume the car to move with a constant velocity due to zero acceleration but in reality it will have acceleration i.e. it speed will fluctuate from time to time. This change in acceleration of that car is the uncertainty/error/variance and we introduce it to our system using process noise.l

Update: In this step we take the **actual measured value** from the devices of

the system. In case of autonomous vehicle these devices can be radar or Lidar. We then calculate the difference between the predicted value and the measured value and then decide which value to keep i.e. predicted value or measured value by calculating the **Kalman Gain.** Based upon the decision made by the Kalman gain we calculate the new value and new uncertainty/error/variance.

Kalman Gain: It determines whether our predicted or measured value is close to the actual value. Its value ranges from 0 to 1. If its value is near to 0 then it means predicted value is close to the actual value or if the value is near to 1 then it means final measured value is close to the actual value. Its value ranges from 0 to 1 because it uses the uncertainty/errors in the predicted and the measured value and is represented by the simple formula as show below. (***K = Error in Prediction / (Error in Prediction + Error in Measurement))***

Error in Measurement *is generally given by the sensor manufacturers. When we buy a new sensor, the manufacturer tells us the standard deviation of the measurement that we'll get from the sensor. It means, let say the standard deviation is 3 and actual measurement is 150, then the sensor can give us the output ranges from 147–153.*
Error in Prediction *is calculated mathematically. We initially start with a wrong belief (large error) and then reduces the error gradually (using Kalman Gain) after taking the first few measurements from the sensor.*
This output from the update step is again fed back to the prediction step and the process continues till the difference between the predicted value and the measured value tend to convert to zero. This calculated value will be the prediction/educated guess done by the Kalman Filter.

Limitations of Kalman Filter:

All these functions used in Kalman Filter are linear as they do not include any angle parameter in their equation therefore we can directly feed these linear function to Gaussian distribution to get a Gaussian distribution mean and variance. But in real life situation most of the problems are non-linear. Due to which we cannot directly feed this non-linear function to the Gaussian distribution as it will lead to non-Gaussian distribution and we won't be able to apply Kalman Filter on it anymore. This problem of non-linearity in

Kalman Filter was resolved using **Extended Kalman Filter.**

What is Extended Kalman Filter?

Extended Kalman Filters (EKF) linearize the distribution around the mean of the current estimate and then use this linearization in predict and update states of the Kalman Filter algorithm.
The predicated state is represented by Gaussian distribution which is mapped to nonlinear function (Radar measurement), then resulting function will not be a Gaussian function. So Kalman filter is not applicable any more as it works with Gaussian functions only. So what does EKF do is it linearizes the resulting function and proceed further accordingly. The EKF does not alter the Kalman filter's linear equations. Instead, it *linearizes* the nonlinear equations at the point of the current estimate, and uses this linearization in the linear Kalman filter.

Implementation of Extended Kalman Filter

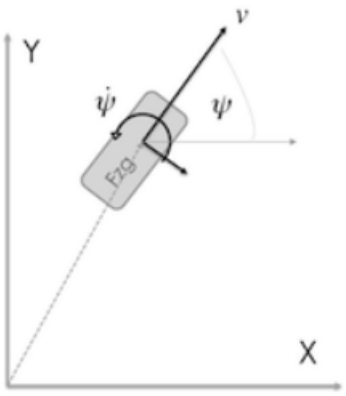
Extended Kalman Filter Implementation for Constant Turn Rate and Velocity (CTRV) Vehicle Model in Python.

The modules used for this code are following:

[1]:
```
import numpy as np
%matplotlib inline
import matplotlib.dates as mdates
import matplotlib.pyplot as plt
from scipy.stats import norm
from sympy import Symbol, symbols, Matrix, sin, cos
from sympy import init_printing
init_printing(use_latex=True)
```

For this project we have a velocity sensor, which measures the vehicle speed (υ) in heading direction (ψ) and a yaw rate sensor (ψ˙) which both have to fused with the position (x & y) from a GPS sensor.

$$x_k = \begin{bmatrix} x \\ y \\ \psi \\ v \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} \text{Position X} \\ \text{Position Y} \\ \text{Heading} \\ \text{Velocity} \\ \text{Yaw Rate} \end{bmatrix}$$



Now storing these variables in code.

```
numstates=5 # States
dt = 1.0/50.0 # Sample Rate of the Measurements is 50Hz
dtGPS=1.0/10.0 # Sample Rate of GPS is 10Hz

#Developing the math behind dynamic model
vs, psis, dpsis, dts, xs, ys, lats, lons = symbols('v \psi \dot\psi T x y lat lon')

gs = Matrix([[xs+(vs/dpsis)*(sin(psis+dpsis*dts)-sin(psis))],
             [ys+(vs/dpsis)*(-cos(psis+dpsis*dts)+cos(psis))],
             [psis+dpsis*dts],
             [vs],
             [dpsis]])
state = Matrix([xs,ys,psis,vs,dpsis])
```

[70]:
```
gs
```

[70]:

$$\begin{bmatrix} x + \frac{v(-\sin(\psi)+\sin(T\dot{\psi}+\psi))}{\dot{\psi}} \\ y + \frac{v(\cos(\psi)-\cos(T\dot{\psi}+\psi))}{\dot{\psi}} \\ T\dot{\psi} + \psi \\ v \\ \dot{\psi} \end{bmatrix}$$

[5]:
```
#Above formulas calculate how the state is evolving from one to the next time step
```

[6]:
```
#Calculate the Jacobian of the Dynamic function  with respect to the state vector x
state
```

[6]:

$$\begin{bmatrix} x \\ y \\ \psi \\ v \\ \dot{\psi} \end{bmatrix}$$

[71]:
```
gs.jacobian(state)
```

[71]:

$$\begin{bmatrix} 1 & 0 & \frac{v(-\cos(\psi)+\cos(T\dot{\psi}+\psi))}{\dot{\psi}} & \frac{-\sin(\psi)+\sin(T\dot{\psi}+\psi)}{\dot{\psi}} & \frac{Tv\cos(T\dot{\psi}+\psi)}{\dot{\psi}} - \frac{v(-\sin(\psi)+\sin(T\dot{\psi}+\psi))}{\dot{\psi}^2} \\ 0 & 1 & \frac{v(-\sin(\psi)+\sin(T\dot{\psi}+\psi))}{\dot{\psi}} & \frac{\cos(\psi)-\cos(T\dot{\psi}+\psi)}{\dot{\psi}} & \frac{Tv\sin(T\dot{\psi}+\psi)}{\dot{\psi}} - \frac{v(\cos(\psi)-\cos(T\dot{\psi}+\psi))}{\dot{\psi}^2} \\ 0 & 0 & 1 & 0 & T \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

[8]:
```
#Initial Uncertainty P0
```

[72]:
```
#Initialized with  means you are pretty sure where the vehicle starts
P = np.diag([1000.0, 1000.0, 1000.0, 1000.0, 1000.0])
print(P, P.shape)
```

```
[[1000.    0.    0.    0.    0.]
 [   0. 1000.    0.    0.    0.]
 [   0.    0. 1000.    0.    0.]
 [   0.    0.    0. 1000.    0.]
 [   0.    0.    0.    0. 1000.]] (5, 5)
```

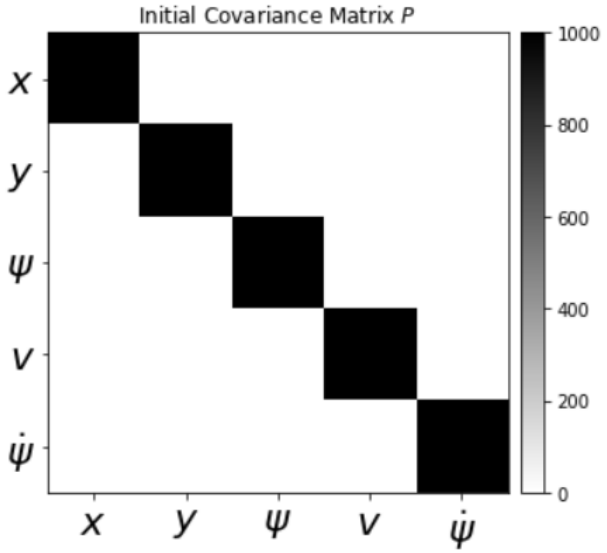
[73]:
```
fig = plt.figure(figsize=(5, 5))
im = plt.imshow(P, interpolation="none", cmap=plt.get_cmap('binary'))
plt.title('Initial Covariance Matrix $P$')
ylocs, ylabels = plt.yticks()
# set the locations of the yticks
plt.yticks(np.arange(6))
# set the locations and labels of the yticks
plt.yticks(np.arange(5),('$x$', '$y$', '$\psi$', '$v$', '$\dot \psi$'), fontsize=22)

xlocs, xlabels = plt.xticks()
# set the locations of the yticks
plt.xticks(np.arange(6))
# set the locations and labels of the yticks
plt.xticks(np.arange(5),('$x$', '$y$', '$\psi$', '$v$', '$\dot \psi$'), fontsize=22)

plt.xlim([-0.5,4.5])
plt.ylim([4.5, -0.5])

from mpl_toolkits.axes_grid1 import make_axes_locatable
divider = make_axes_locatable(plt.gca())
cax = divider.append_axes("right", "5%", pad="3%")
plt.colorbar(im, cax=cax)


plt.tight_layout()
```



[11]:
```
#Process Noise Covariance Matrix Q
```

[74]:
```
sGPS     = 0.5*8.8*dt**2  # assume 8.8m/s2 as maximum acceleration, forcing the vehicle
sCourse  = 0.1*dt # assume 0.1rad/s as maximum turn rate for the vehicle
sVelocity= 8.8*dt # assume 8.8m/s2 as maximum acceleration, forcing the vehicle
sYaw     = 1.0*dt # assume 1.0rad/s2 as the maximum turn rate acceleration for the vehicle

Q = np.diag([sGPS**2, sGPS**2, sCourse**2, sVelocity**2, sYaw**2])
print(Q, Q.shape)
```

```
[[3.0976e-06 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00]
 [0.0000e+00 3.0976e-06 0.0000e+00 0.0000e+00 0.0000e+00]
 [0.0000e+00 0.0000e+00 4.0000e-06 0.0000e+00 0.0000e+00]
 [0.0000e+00 0.0000e+00 0.0000e+00 3.0976e-02 0.0000e+00]
 [0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 4.0000e-04]] (5, 5)
```

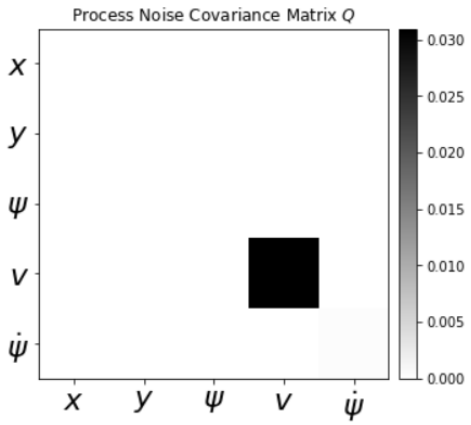
[75]:
```
fig = plt.figure(figsize=(5, 5))
im = plt.imshow(Q, interpolation="none", cmap=plt.get_cmap('binary'))
plt.title('Process Noise Covariance Matrix $Q$')
ylocs, ylabels = plt.yticks()
# set the locations of the yticks
plt.yticks(np.arange(6))
# set the locations and labels of the yticks
plt.yticks(np.arange(5),('$x$', '$y$', '$\psi$', '$v$', '$\dot \psi$'), fontsize=22)

xlocs, xlabels = plt.xticks()

# set the locations of the yticks
plt.xticks(np.arange(6))
# set the locations and labels of the yticks
plt.xticks(np.arange(5),('$x$', '$y$', '$\psi$', '$v$', '$\dot \psi$'), fontsize=22)

plt.xlim([-0.5,4.5])
plt.ylim([4.5, -0.5])

from mpl_toolkits.axes_grid1 import make_axes_locatable
divider = make_axes_locatable(plt.gca())
cax = divider.append_axes("right", "5%", pad="3%")
plt.colorbar(im, cax=cax);
```



Loading the dataset which contains the value of data collected from lidar and radar.

```
#path = './../RaspberryPi-CarPC/TinkerDataLogger/DataLogs/2014/'
datafile = '2014-03-26-000-Data.csv'

date, \
time, \
millis, \
ax, \
ay, \
az, \
rollrate, \
pitchrate, \
yawrate, \
roll, \
pitch, \
yaw, \
speed, \
course, \
latitude, \
longitude, \
altitude, \
pdop, \
hdop, \
vdop, \
epe, \
fix, \
satellites_view, \
satellites_used, \
temp = np.loadtxt(datafile, delimiter=',', unpack=True,
                  converters={1: mdates.strpdate2num('%H%M%S%f'),
                              0: mdates.strpdate2num('%y%m%d')},
                  skiprows=1)

print('Read \'%s\' successfully.' % datafile)

# A course of 0° means the Car is traveling north bound
# and 90° means it is traveling east bound.
# In the Calculation following, East is Zero and North is 90°
# We need an offset.
course =(-course+90.0)
```

```
Read '2014-03-26-000-Data.csv' successfully.
```

[78]:
```
#Measurement Function h
#Matrix JH is the Jacobian of the Measurement function h with respect to the state.
#Function h can be used to compute the predicted measurement from the predicted state.
```

[79]:
```
#If a GPS measurement is available, the following function maps the state to the measurement
hs = Matrix([[xs],
             [ys],
             [vs],
             [dpsis]])
hs
```

[79]:
$$\begin{bmatrix} x \\ y \\ v \\ \dot{\psi} \end{bmatrix}$$

[80]:
```
JHs=hs.jacobian(state)
JHs
```

[80]:
$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

[81]:
```
#If no GPS measurement is available, simply set the corresponding values in Jh to zero.
```

[82]:
```
#Measurement Noise Covariance  R
```

[83]:
```
varGPS = 6.0 # Standard Deviation of GPS Measurement
varspeed = 1.0 # Variance of the speed measurement
varyaw = 0.1 # Variance of the yawrate measurement
R = np.matrix([[varGPS**2, 0.0, 0.0, 0.0],
               [0.0, varGPS**2, 0.0, 0.0],
               [0.0, 0.0, varspeed**2, 0.0],
               [0.0, 0.0, 0.0, varyaw**2]])

print(R, R.shape)
```

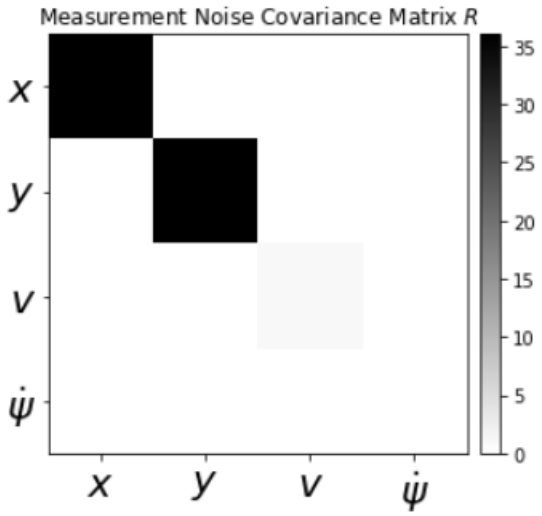
```
[[3.6e+01 0.0e+00 0.0e+00 0.0e+00]
 [0.0e+00 3.6e+01 0.0e+00 0.0e+00]
 [0.0e+00 0.0e+00 1.0e+00 0.0e+00]
 [0.0e+00 0.0e+00 0.0e+00 1.0e-02]] (4, 4)
```

[84]:
```
fig = plt.figure(figsize=(4.5, 4.5))
im = plt.imshow(R, interpolation="none", cmap=plt.get_cmap('binary'))
plt.title('Measurement Noise Covariance Matrix $R$')
ylocs, ylabels = plt.yticks()
# set the locations of the yticks
plt.yticks(np.arange(5))
# set the locations and labels of the yticks
plt.yticks(np.arange(4),('$x$', '$y$', '$v$', '$\dot \psi$'), fontsize=22)

xlocs, xlabels = plt.xticks()
# set the locations of the yticks
plt.xticks(np.arange(5))
# set the locations and labels of the yticks
plt.xticks(np.arange(4),('$x$', '$y$', '$v$', '$\dot \psi$'), fontsize=22)

plt.xlim([-0.5,3.5])
plt.ylim([3.5, -0.5])

from mpl_toolkits.axes_grid1 import make_axes_locatable
divider = make_axes_locatable(plt.gca())
cax = divider.append_axes("right", "5%", pad="3%")
plt.colorbar(im, cax=cax);
```



[85]:
```
#identity matrix
I = np.eye(numstates)
print(I, I.shape)
```

```
[[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]] (5, 5)
```

[86]:
```
#Approx. Lat/Lon to Meters to check Location
RadiusEarth = 6378388.0 # m
arc= 2.0*np.pi*(RadiusEarth+altitude)/360.0 # m/°

dx = arc * np.cos(latitude*np.pi/180.0) * np.hstack((0.0, np.diff(longitude))) # in m
dy = arc * np.hstack((0.0, np.diff(latitude))) # in m

mx = np.cumsum(dx)
my = np.cumsum(dy)

ds = np.sqrt(dx**2+dy**2)

GPS=(ds!=0.0).astype('bool') # GPS Trigger for Kalman Filter
```

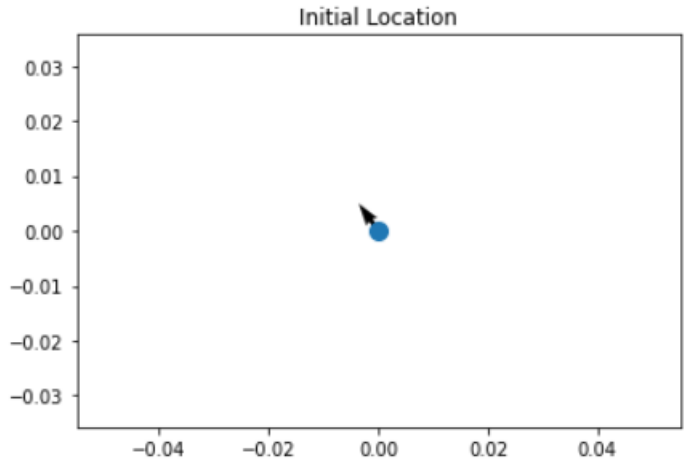
[87]:
```
#Initial State
x = np.matrix([[mx[0], my[0], course[0]/180.0*np.pi, speed[0]/3.6+0.001, yawrate[0]/180.0*np.pi]]).T
print(x, x.shape)

U=float(np.cos(x[2])*x[3])
V=float(np.sin(x[2])*x[3])

plt.quiver(x[0], x[1], U, V)
plt.scatter(float(x[0]), float(x[1]), s=100)
plt.title('Initial Location')
plt.axis('equal')
```

```
[[ 0.        ]
 [ 0.        ]
 [-4.08756111]
 [ 0.67322222]
 [-0.32660346]] (5, 1)
```

[87]: $(-0.055,\ 0.055,\ -0.055,\ 0.055)$



[88]:
```
#Put everything together as a measurement vector
measurements = np.vstack((mx, my, speed/3.6, yawrate/180.0*np.pi))
# Lenth of the measurement
m = measurements.shape[1]
print(measurements.shape)
```

```
(4, 10800)
```

[89]:
```
# Preallocation for Plotting
x0 = []
x1 = []
x2 = []
x3 = []
x4 = []
x5 = []
Zx = []
Zy = []
Px = []
Py = []
Pdx= []
Pdy= []
Pddx=[]
Pddy=[]
Kx = []
Ky = []
Kdx= []
Kdy= []
Kddx=[]
dstate=[]


def savestates(x, Z, P, K):
    x0.append(float(x[0]))
    x1.append(float(x[1]))
    x2.append(float(x[2]))
    x3.append(float(x[3]))
    x4.append(float(x[4]))
    Zx.append(float(Z[0]))
    Zy.append(float(Z[1]))
    Px.append(float(P[0,0]))
    Py.append(float(P[1,1]))
    Pdx.append(float(P[2,2]))
    Pdy.append(float(P[3,3]))
    Pddx.append(float(P[4,4]))
    Kx.append(float(K[0,0]))
    Ky.append(float(K[1,0]))
    Kdx.append(float(K[2,0]))
    Kdy.append(float(K[3,0]))
    Kddx.append(float(K[4,0]))
```

[90]:
```
#Extended Kalman Filter
```

[91]:
```
for filterstep in range(m):

    # Time Update (Prediction)
    # ========================
    # Project the state ahead
    # see "Dynamic Matrix"
    if np.abs(yawrate[filterstep])<0.0001: # Driving straight
        x[0] = x[0] + x[3]*dt * np.cos(x[2])
        x[1] = x[1] + x[3]*dt * np.sin(x[2])
        x[2] = x[2]
        x[3] = x[3]
        x[4] = 0.0000001 # avoid numerical issues in Jacobians
        dstate.append(0)
    else: # otherwise
        x[0] = x[0] + (x[3]/x[4]) * (np.sin(x[4]*dt+x[2]) - np.sin(x[2]))
        x[1] = x[1] + (x[3]/x[4]) * (-np.cos(x[4]*dt+x[2])+ np.cos(x[2]))
        x[2] = (x[2] + x[4]*dt + np.pi) % (2.0*np.pi) - np.pi
        x[3] = x[3]
        x[4] = x[4]
        dstate.append(1)

    # Calculate the Jacobian of the Dynamic Matrix A
    # see "Calculate the Jacobian of the Dynamic Matrix with respect to the state vector"
    a13 = float((x[3]/x[4]) * (np.cos(x[4]*dt+x[2]) - np.cos(x[2])))
    a14 = float((1.0/x[4]) * (np.sin(x[4]*dt+x[2]) - np.sin(x[2])))
    a15 = float((dt*x[3]/x[4])*np.cos(x[4]*dt+x[2]) - (x[3]/x[4]**2)*(np.sin(x[4]*dt+x[2]) - np.sin(x[2])))
    a23 = float((x[3]/x[4]) * (np.sin(x[4]*dt+x[2]) - np.sin(x[2])))
    a24 = float((1.0/x[4]) * (-np.cos(x[4]*dt+x[2]) + np.cos(x[2])))
    a25 = float((dt*x[3]/x[4])*np.sin(x[4]*dt+x[2]) - (x[3]/x[4]**2)*(-np.cos(x[4]*dt+x[2]) + np.cos(x[2])))
    JA = np.matrix([[1.0, 0.0, a13, a14, a15],
                    [0.0, 1.0, a23, a24, a25],
                    [0.0, 0.0, 1.0, 0.0, dt],
                    [0.0, 0.0, 0.0, 1.0, 0.0],
                    [0.0, 0.0, 0.0, 0.0, 1.0]])


    # Project the error covariance ahead
    P = JA*P*JA.T + Q
```

```
    # Measurement Update (Correction)
    # ===============================
    # Measurement Function
    hx = np.matrix([[float(x[0])],
                    [float(x[1])],
                    [float(x[3])],
                    [float(x[4])]])

    if GPS[filterstep]: # with 10Hz, every 5th step
        JH = np.matrix([[1.0, 0.0, 0.0, 0.0, 0.0],
                        [0.0, 1.0, 0.0, 0.0, 0.0],
                        [0.0, 0.0, 0.0, 1.0, 0.0],
                        [0.0, 0.0, 0.0, 0.0, 1.0]])
    else: # every other step
        JH = np.matrix([[0.0, 0.0, 0.0, 0.0, 0.0],
                        [0.0, 0.0, 0.0, 0.0, 0.0],
                        [0.0, 0.0, 0.0, 1.0, 0.0],
                        [0.0, 0.0, 0.0, 0.0, 1.0]])

    S = JH*P*JH.T + R
    K = (P*JH.T) * np.linalg.inv(S)

    # Update the estimate via
    Z = measurements[:,filterstep].reshape(JH.shape[0],1)
    y = Z - (hx)                         # Innovation or Residual
    x = x + (K*y)

    # Update the error covariance
    P = (I - (K*JH))*P

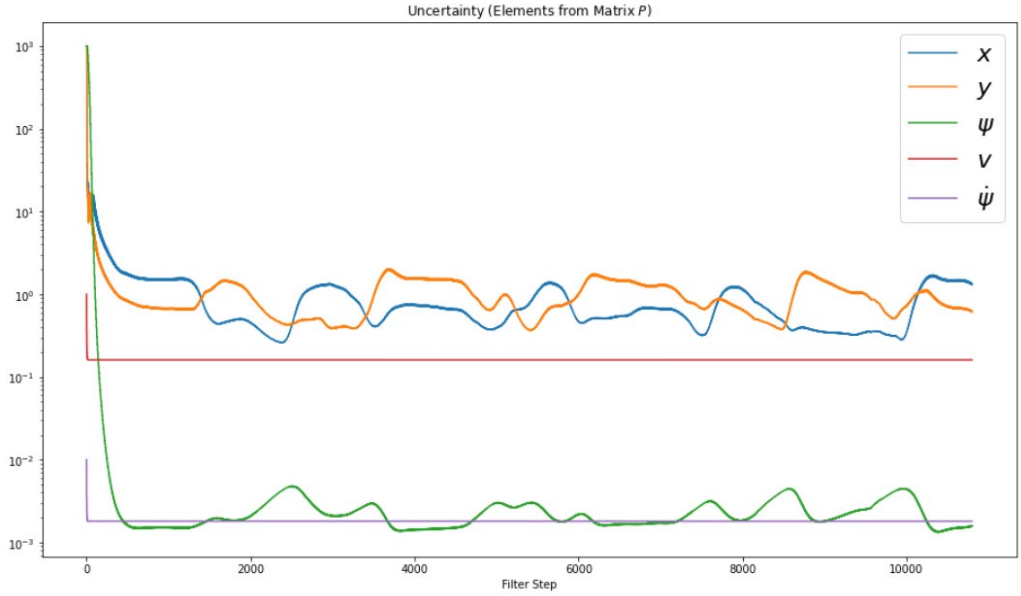


    # Save states for Plotting
    savestates(x, Z, P, K)
```

[92]:
```
#Lets take a look at the filter performance
def plotP():
    fig = plt.figure(figsize=(16,9))
    plt.semilogy(range(m),Px, label='$x$')
    plt.step(range(m),Py, label='$y$')
    plt.step(range(m),Pdx, label='$\psi$')
    plt.step(range(m),Pdy, label='$v$')
    plt.step(range(m),Pddx, label='$\dot \psi$')

    plt.xlabel('Filter Step')
    plt.ylabel('')
    plt.title('Uncertainty (Elements from Matrix $P$)')
    plt.legend(loc='best',prop={'size':22})
```

[93]:
```
#Uncertainties in
plotP()
```



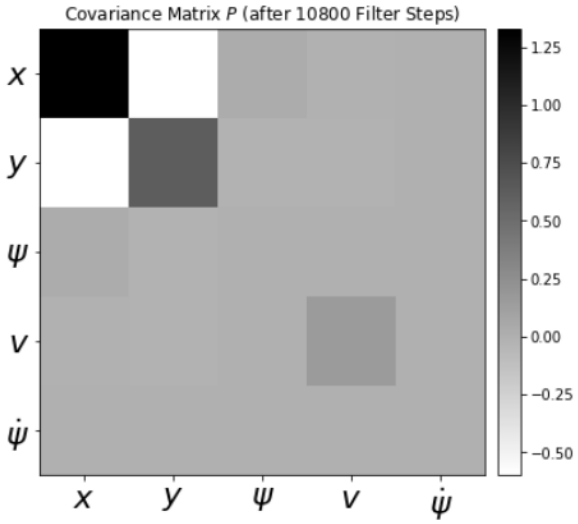
[94]:
```
fig = plt.figure(figsize=(6, 6))
im = plt.imshow(P, interpolation="none", cmap=plt.get_cmap('binary'))
plt.title('Covariance Matrix $P$ (after %i Filter Steps)' % (m))
ylocs, ylabels = plt.yticks()
# set the locations of the yticks
plt.yticks(np.arange(6))
# set the locations and labels of the yticks
plt.yticks(np.arange(5),('$x$', '$y$', '$\psi$', '$v$', '$\dot \psi$'), fontsize=22)

xlocs, xlabels = plt.xticks()
# set the locations of the yticks
plt.xticks(np.arange(6))
# set the locations and labels of the yticks
plt.xticks(np.arange(5),('$x$', '$y$', '$\psi$', '$v$', '$\dot \psi$'), fontsize=22)

plt.xlim([-0.5,4.5])
plt.ylim([4.5, -0.5])

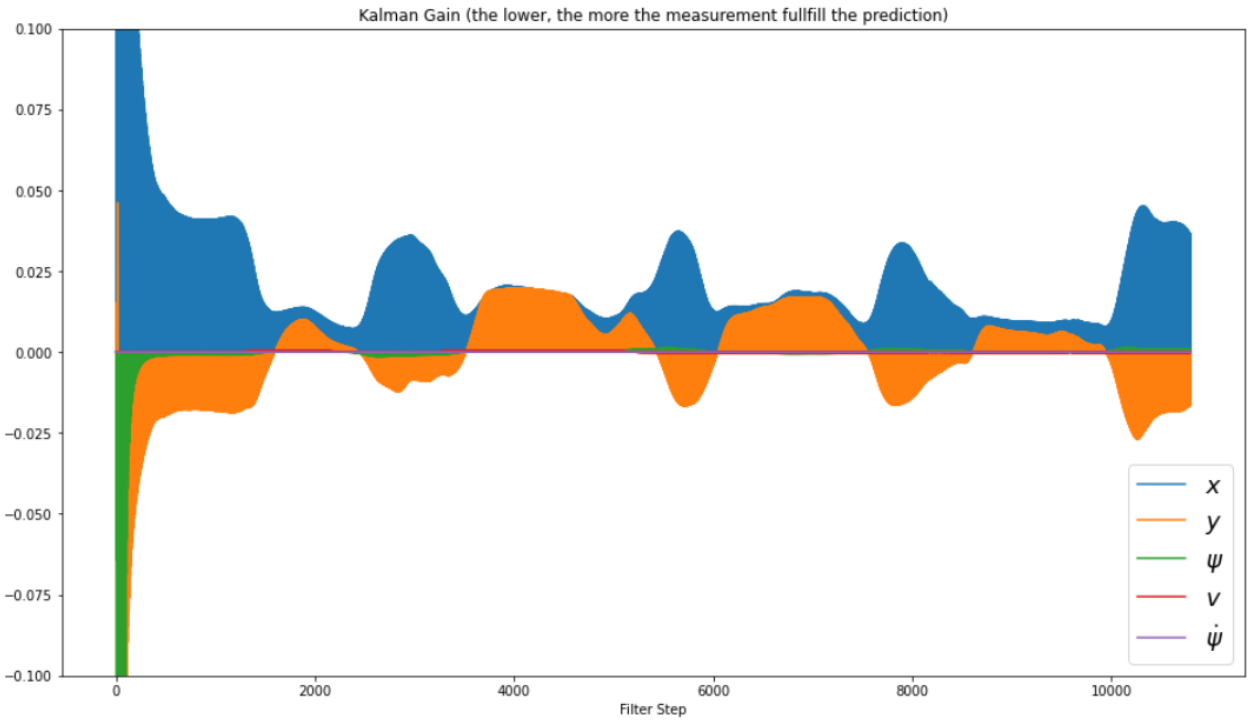
from mpl_toolkits.axes_grid1 import make_axes_locatable
divider = make_axes_locatable(plt.gca())
cax = divider.append_axes("right", "5%", pad="3%")
plt.colorbar(im, cax=cax)
```

[94]: <matplotlib.colorbar.Colorbar at 0x120c20ce3a0>



[95]:
```
fig = plt.figure(figsize=(16,9))
plt.step(range(len(measurements[0])),Kx, label='$x$')
plt.step(range(len(measurements[0])),Ky, label='$y$')
plt.step(range(len(measurements[0])),Kdx, label='$\psi$')
plt.step(range(len(measurements[0])),Kdy, label='$v$')
plt.step(range(len(measurements[0])),Kddx, label='$\dot \psi$')


plt.xlabel('Filter Step')
plt.ylabel('')
plt.title('Kalman Gain (the lower, the more the measurement fullfill the prediction)')
plt.legend(prop={'size':18})
plt.ylim([-0.1,0.1]);
```



[96]:
```
#State Vector
def plotx():
    fig = plt.figure(figsize=(16,16))

    plt.subplot(411)
    plt.step(range(len(measurements[0])),x0-mx[0], label='$x$')
    plt.step(range(len(measurements[0])),x1-my[0], label='$y$')

    plt.title('Extended Kalman Filter State Estimates (State Vector $x$)')
    plt.legend(loc='best',prop={'size':22})
    plt.ylabel('Position (relative to start) [m]')

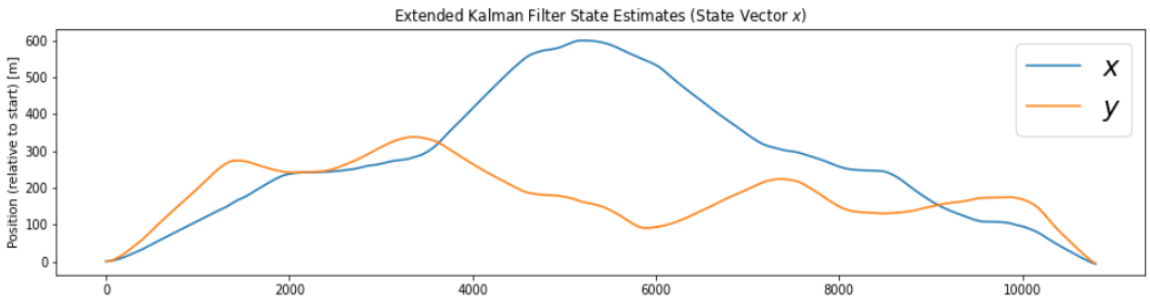
    plt.subplot(412)
    plt.step(range(len(measurements[0])),x2, label='$\psi$')
    plt.step(range(len(measurements[0])),(course/180.0*np.pi+np.pi)%(2.0*np.pi) - np.pi, label='$\psi$ (from GPS as reference)')
    plt.ylabel('Course')
    plt.legend(loc='best',prop={'size':16})
```

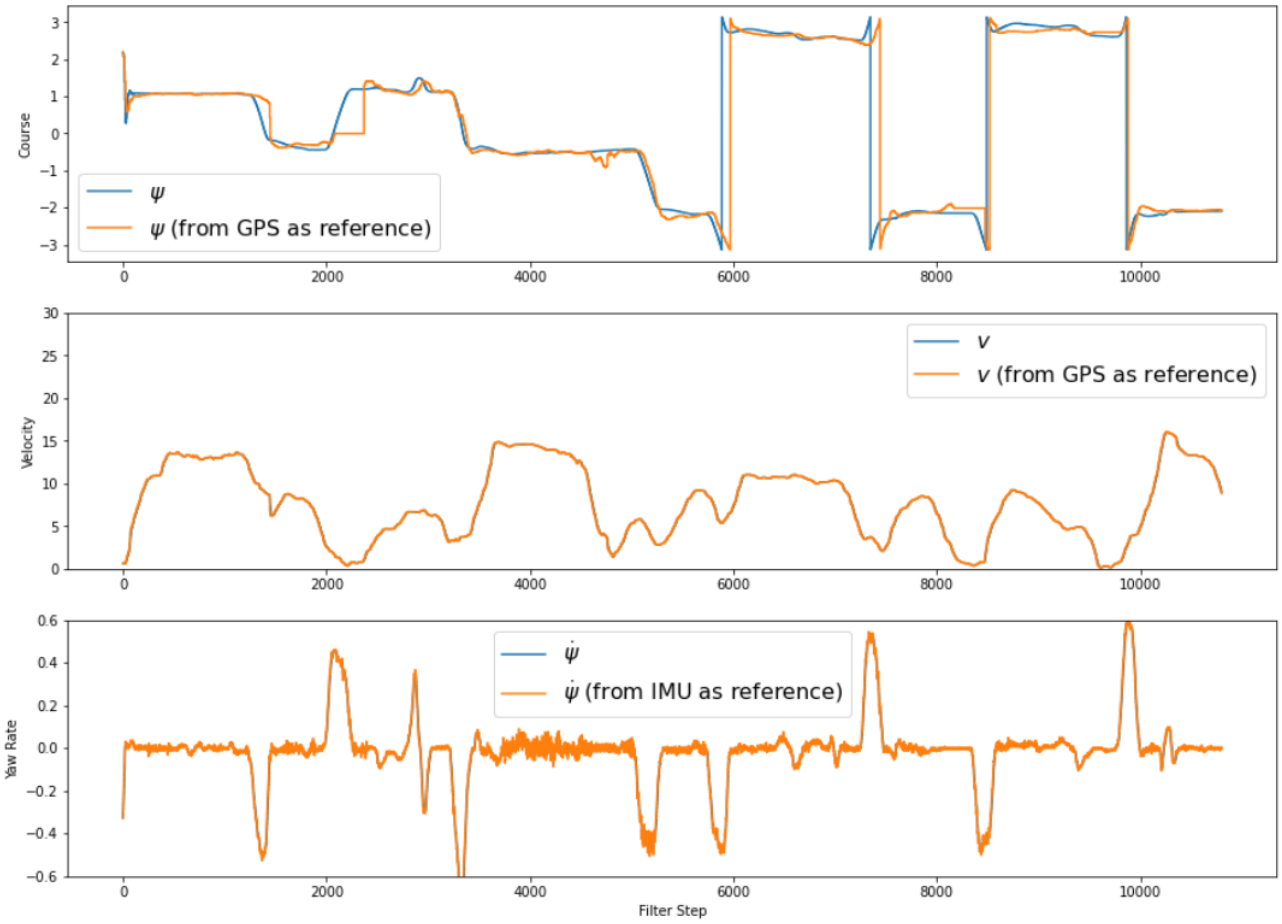
```
    plt.subplot(413)
    plt.step(range(len(measurements[0])),x3, label='$v$')
    plt.step(range(len(measurements[0])),speed/3.6, label='$v$ (from GPS as reference)')
    plt.ylabel('Velocity')
    plt.ylim([0, 30])
    plt.legend(loc='best',prop={'size':16})

    plt.subplot(414)
    plt.step(range(len(measurements[0])),x4, label='$\dot \psi$')
    plt.step(range(len(measurements[0])),yawrate/180.0*np.pi, label='$\dot \psi$ (from IMU as reference)')
    plt.ylabel('Yaw Rate')
    plt.ylim([-0.6, 0.6])
    plt.legend(loc='best',prop={'size':16})
    plt.xlabel('Filter Step')

    plt.savefig('Extended-Kalman-Filter-CTRV-State-Estimates.png', dpi=72, transparent=True, bbox_inches='tight')
```

[97]:
```
plotx()
```





[98]:
```
#%pylab --no-import-all
```

[99]:
```
def plotxy():

    fig = plt.figure(figsize=(16,9))

    # EKF State
    plt.quiver(x0,x1,np.cos(x2), np.sin(x2), color='#94C600', units='xy', width=0.05, scale=0.5)
    plt.plot(x0,x1, label='EKF Position', c='k', lw=5)

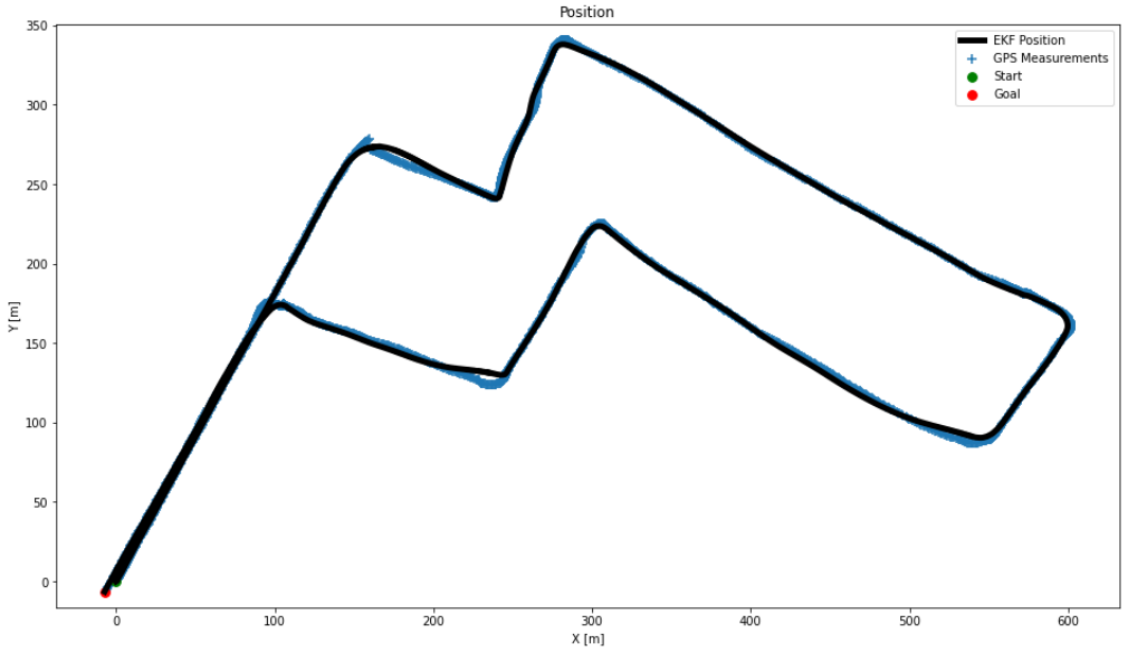
    # Measurements
    plt.scatter(mx[::5],my[::5], s=50, label='GPS Measurements', marker='+')
    #cbar=plt.colorbar(ticks=np.arange(20))
    #cbar.ax.set_ylabel(u'EPE', rotation=270)
    #cbar.ax.set_xlabel(u'm')

    # Start/Goal
    plt.scatter(x0[0],x1[0], s=60, label='Start', c='g')
    plt.scatter(x0[-1],x1[-1], s=60, label='Goal', c='r')

    plt.xlabel('X [m]')
    plt.ylabel('Y [m]')
    plt.title('Position')
    plt.legend(loc='best')
    plt.axis('equal')
    #plt.tight_layout()

    #plt.savefig('Extended-Kalman-Filter-CTRV-Position.png', dpi=72, transparent=True, bbox_inches='tight')
```

```
[100]: plotxy()
```



Learnings and Conclusion:

The Kalman filter is relatively quick and easy to implement and provides an optimal estimate of the condition for normally distributed noisy sensor values under certain conditions. But there is certain limitation to kalman filter as it cannot be used for non-linear function and so, Extended Kalman Filter is introduced to overcome this challenge by linearizing the nonlinear function of current estimate and then using the same kalman filter algorithm to process further.

References

- https://medium.com/@akhilsuri194/building-an-autonomous-vehicle-part-4-1-sensor-fusion-and-object-tracking-using-kalman-filters-66589e522965
- https://medium.com/@akhilsuri194/building-an-autonomous-vehicle-part-4-2-8049cda5a92
- https://medium.com/@akhilsuri194/building-an-autonomous-vehicle-part-4-3-2a506e57d830
- https://www.kalmanfilter.net/default.aspx
- https://thekalmanfilter.com/kalman-filter-explained-simply/
- https://www.bzarg.com/p/how-a-kalman-filter-works-in-pictures/
- https://towardsdatascience.com/kalman-filter-an-algorithm-for-making-sense-from-the-insights-of-various-sensors-fused-together-ddf67597f35e
- https://blog.quantinsti.com/kalman-filter/